

Slide 1: Combining Patterns

What Are Combined Patterns?

Not every pattern can be created using a single simple loop structure. Some patterns are made by **combining two or more smaller patterns**.

For example:

```
*
**
***
****
***
**
*
```

This pattern can be divided into two parts:

```
*
**
***
****
```

and

```
***
**
*
```

The first part is an **increasing pattern**, while the second part is a **decreasing pattern**.

How Do We Build a Combined Pattern?

A useful approach is:

1. Identify the first part of the pattern.
2. Identify the second part of the pattern.
3. Find the logic used by each part.
4. Write the loops for each part.
5. Combine both parts into one program.

Important Idea

A complex pattern does not always require completely new logic.

Often, it is simply:

Simple Pattern + Simple Pattern = Complex Pattern

For example:

Increasing Triangle

+

Decreasing Triangle

=

Combined Pattern

What Should You Look For?

When analyzing a combined pattern, ask:

- Where does the first shape end?
- Where does the second shape begin?
- Does the number of symbols increase or decrease?
- Do spaces change?
- Does the printed value change?

Key takeaway: Break a large pattern into smaller, understandable sections before writing the C program.

Slide 2: Diamond Pattern

What Is a Diamond Pattern?

A diamond pattern is a combination of an **upper pyramid** and a **lower inverted pyramid**.

Example:

```
*
***
*****
*****
*****
***
*
```

Instead of trying to solve the entire pattern at once, divide it into two parts.

Part 1: Upper Pyramid

```
*  
***  
*****  
*****
```

The number of stars increases in every row.

The number of spaces decreases as the pattern moves downward.

Part 2: Lower Inverted Pyramid

```
*****  
***  
*
```

The number of stars decreases in every row.

The number of spaces increases as the pattern moves downward.

How Does the Diamond Work?

For the upper half:

- Spaces decrease.
- Stars increase.
- Stars appear in odd numbers: 1, 3, 5, 7...

For the lower half:

- Spaces increase.
- Stars decrease.
- Stars continue in reverse order: 5, 3, 1...

Important Concept

The diamond is not a completely new pattern.

It is:

Upper Pyramid

+

Lower Inverted Pyramid

=

Diamond

This teaches an important programming skill: **solving a large problem by dividing it into smaller problems.**

Key takeaway

When a pattern looks complicated, do not immediately try to write one giant loop.

First ask:

"Can I divide this pattern into smaller patterns that I already understand?"

Slide 3: Number Pyramid Patterns

What Is a Number Pyramid?

A number pyramid uses numbers instead of symbols while maintaining a pyramid-shaped structure.

Example:

```
1
123
12345
1234567
```

This pattern has three important components:

1. **Spaces**
2. **Numbers**
3. **Number of values printed in each row**

Understanding these components makes the pattern much easier to program.

Understanding the Spaces

The spaces are used to move the numbers toward the center.

For example:

```
1
 123
12345
```

1234567

The number of spaces decreases as the row number increases.

For 4 rows:

Row 1 → 3 spaces

Row 2 → 2 spaces

Row 3 → 1 space

Row 4 → 0 spaces

This can be represented as:

Spaces = $n - i$

where:

- n = total number of rows
- i = current row

Understanding the Numbers

The number of values increases by 2 for every row:

1

123

12345

1234567

The number of values is:

1, 3, 5, 7...

This is an **odd-number sequence**.

The number of values can be represented as:

$2 * i - 1$

Important Observation

The pattern can be separated into independent questions:

- How many spaces should I print?
- How many numbers should I print?
- Which numbers should I print?

Once these questions are answered, the nested loops become much easier to design.

Key takeaway

A pattern is usually easier to solve when you analyze **each part of a row separately** instead of looking at the complete output as one large shape.

Slide 4: How to Analyze Any Pattern

Don't Start With Code

When students see a pattern, the first instinct is often to immediately start writing `for` loops.

That is usually where the suffering begins.

Before writing code, **analyze the pattern first**.

Step 1: Count the Rows

First determine how many rows the pattern contains.

For example:

```
*  
**  
***  
****  
*****
```

There are **5 rows**.

This usually becomes the outer loop.

Outer loop → controls rows

Step 2: Analyze Each Row

Look at what changes from one row to the next.

Ask:

- How many spaces are there?

- How many stars are there?
- How many numbers are there?
- Does the value change?
- Does the number of characters increase or decrease?

Step 3: Count Spaces

For aligned patterns, spaces are part of the pattern.

Example:

```
*
***
*****
*****
```

The spaces change from row to row.

Determine their relationship with the row number.

Step 4: Count Symbols or Numbers

Determine how many characters are printed in each row.

For example:

```
*
***
*****
*****
```

The number of stars is:

1, 3, 5, 7

This gives us a mathematical relationship.

Step 5: Identify What Changes

Look for patterns such as:

- Increasing values
- Decreasing values
- Repeated values
- Odd-number sequences
- Even-number sequences

- Changing spaces
- Changing characters

Step 6: Find the Formula or Rule

Once the relationship is clear, express it as a rule.

For example:

Stars = $2 * i - 1$

or:

Spaces = $n - i$

Step 7: Convert the Logic Into Loops

Finally, translate the rules into C:

Outer loop

↓

Print spaces

↓

Print symbols/numbers

↓

Move to next line

Key takeaway

Analyze first. Code second.

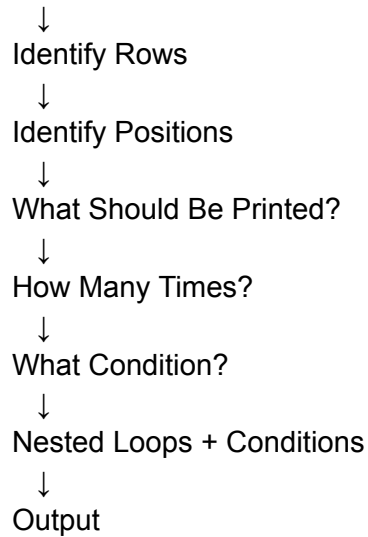
The loops should be the result of your analysis, not the starting point.

Slide 5: Pattern Problem-Solving Framework

A General Method for Solving Patterns

Most pattern problems can be solved using the same basic thinking process.

Pattern



1. Identify the Rows

Determine how many rows the pattern contains.

The **outer loop** usually controls the rows.

```
for (i = 1; i <= n; i++)
```

2. Identify Positions

Think about the position of every character.

A character has a:

- Row position
- Column position

For example:

```
* . . *  
. * * .  
. * * .  
. * * .  
* . . *
```

The decision to print * depends on its position.

3. Decide What to Print

At every position, determine whether you should print:

- *
- A number
- A character
- A space

This is where the inner loop becomes important.

4. Determine How Many Times

Ask:

How many characters should appear in this row?

The answer may depend on:

- Current row
- Total rows
- Current column
- A mathematical formula

5. Identify Conditions

Some patterns require decisions.

For example:

```
if (i == 1 || i == n)
```

could identify the first and last rows of a hollow pattern.

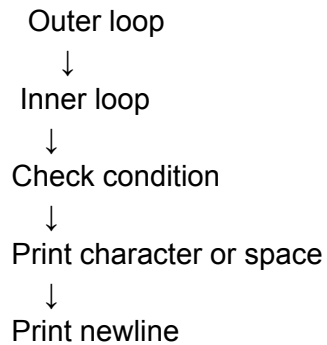
Other patterns may use conditions such as:

```
if (i == j)
```

to identify a diagonal.

6. Build the Nested Loops

The final structure often becomes:



The Most Important Rule

Do not memorize the code for every pattern.

Instead, learn to answer these questions:

1. How many rows are there?
2. What changes from row to row?
3. How many spaces are required?
4. How many characters are required?
5. What should be printed at each position?
6. Is there a condition controlling the output?
7. Can the pattern be divided into smaller patterns?

Final takeaway

Pattern programming is not really about stars, numbers, or pyramids.

It is about learning how to:

Observe → Analyze → Find the Rule → Build the Loops → Produce the Output

Questions:

Question 1: Conceptual MCQ

What is the best approach when a pattern looks complex?

- A. Write one large loop immediately
- B. Memorize a similar program
- C. Break the pattern into smaller, understandable patterns
- D. Avoid using nested loops

 **Hint:** Complex patterns are often combinations of simpler patterns.

 **Glossary:**

- **Combined pattern:** A pattern formed by joining two or more smaller patterns.
- **Nested loop:** A loop inside another loop.

 **Answer: C. Break the pattern into smaller, understandable patterns**

Question 2: Code & Output

What will be the output of the following C program?

```
#include <stdio.h>
```

```
int main() {  
    int i;  
  
    for (i = 1; i <= 4; i++)  
        printf("*");  
  
    printf("\n");  
  
    for (i = 3; i >= 1; i--)  
        printf("*");  
  
    return 0;  
}
```

A.

```
****  
***
```

B.

```
****  
**
```

C.

```
***  
****
```

D.

```
*****
```



Hint: The first loop prints 4 stars. The second loop prints 3 stars.



Glossary:

- `i++`: Increases `i` by 1.
- `i--`: Decreases `i` by 1.
- **Increasing pattern**: Number of symbols grows.
- **Decreasing pattern**: Number of symbols becomes smaller.



Answer: A

Question 3: Fill in the Blank

Complete the code to print the upper part of a number pyramid with an odd number of values in each row.

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= 2 * i - 1; j++) {  
        printf("%d", j);  
    }  
    printf("\n");  
}
```

How many values are printed in row `i`?

2 * ____ - 1

 **Hint:** The number of values follows 1, 3, 5, 7....

 **Glossary:**

- **Odd-number sequence:** A sequence such as 1, 3, 5, 7.
- **i:** Current row number.

 **Answer:** i

Question 4: Fill in the Blank

Complete the code to print the spaces before the numbers in a number pyramid.

```
for (int i = 1; i <= n; i++) {  
  
    for (int j = 1; j <= n - i; j++) {  
        printf(" ");  
    }  
  
    for (int j = 1; j <= 2 * i - 1; j++) {  
        printf("%d", j);  
    }  
  
    printf("\n");  
}
```

The number of spaces in each row is:

n - ____

 **Hint:** As the row number increases, the number of spaces decreases.

 **Glossary:**

- **Alignment:** Positioning the pattern using spaces.
- **Current row:** The row currently being processed by the outer loop.

 **Answer:** i

Question 5: Fill in the Blank

A diamond can be created by combining an upper pyramid with a lower inverted pyramid.

Complete the condition for the lower half so that the number of rows decreases from 5 to 1.

```
for (int i = n - 1; i >= 1; i--) {  
  
    for (int j = 1; j <= n - i; j++) {  
        printf(" ");  
    }  
  
    for (int j = 1; j <= 2 * i - 1; j++) {  
        printf("*");  
    }  
  
    printf("\n");  
}
```

The outer loop should start from:

```
for (int i = ____; i >= 1; i--)
```

 **Hint:** The lower half begins after the widest row, so it starts from one row less than **n**.

 **Glossary:**

- **Inverted pyramid:** A pattern where the number of symbols decreases row by row.
- **Diamond:** An upper pyramid combined with a lower inverted pyramid.

 **Answer:** **n - 1**